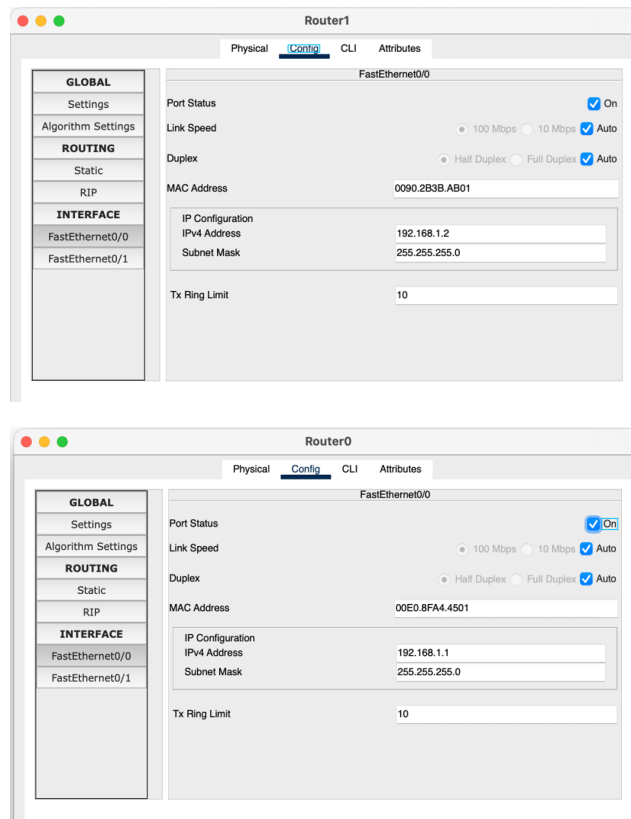


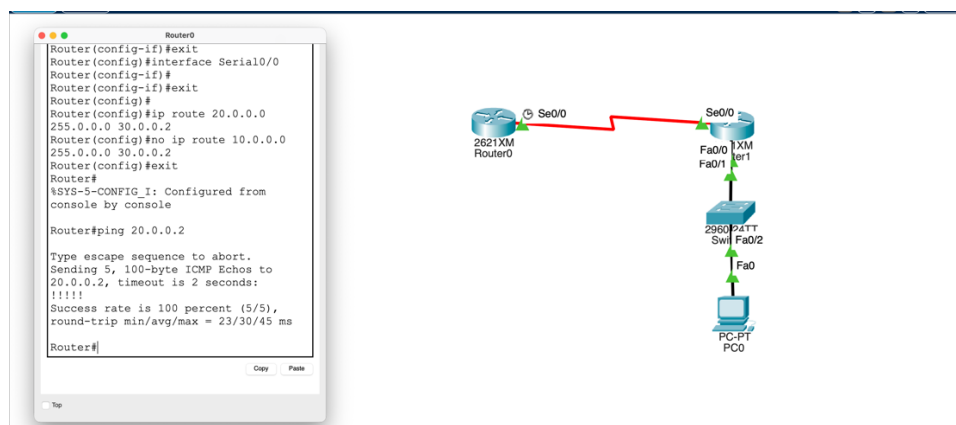
Day 1 — Assignment

1. Open Cisco Packet Tracer and set up a simple network with two routers connected via a serial cable, assigning IP addresses to both router interfaces



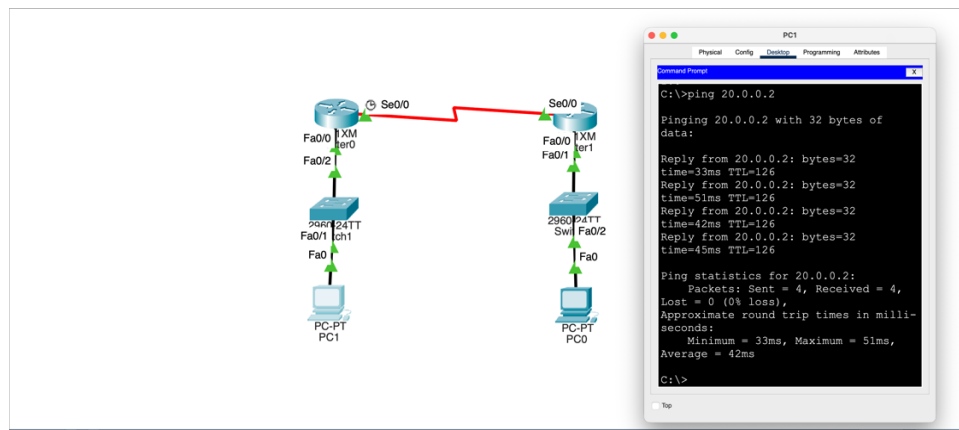
Router1 and Router2 serial interfaces configured with IP addresses.

2. On Router1, configure a static route so it can reach a PC connected to Router2's LAN network. Test connectivity using ping from Router1 to the PC.



Static route configured on Router1; ping to Router2's LAN PC succeeds.

3. Simulate a Zomato-style delivery network (Router1 = Ahmedabad branch, Router2 = Surat branch); configure static routes so each city's LAN can communicate.



Static routes configured between the Ahmedabad and Surat branch LANs.

4. Two advantages and two disadvantages of static routing vs dynamic routing, using IRCTC/Swiggy examples

Advantages of Static Routing

1. More control: routes are manually configured, so the network administrator has full control over the path. For instance, IRCTC can use static routing for booking traffic between specific servers.
2. Lower network overhead: static routes don't exchange routing information, so they consume less bandwidth — helping an app like IRCTC keep predictable, low-overhead connectivity.

Disadvantages of Static Routing

1. No auto-update: if a route fails, the router won't automatically find an alternate path. Swiggy could see delays in delivery-status updates reaching its servers.
2. Hard to scale: every route must be configured individually. If Swiggy adds many new city servers, manually configuring each route becomes increasingly impractical.

Day 2 — Introduction to Dynamic Routing

Theory: What is dynamic routing? Comparison between static and dynamic routing. Practical: Observe RIP routing updates in a pre-configured network.

1. Three differences between static and dynamic routing (table, with real-world analogies)

Static Routing	Dynamic Routing
Routes are configured manually by the administrator — like manually telling a delivery rider the exact street route to take every time.	Routes are learned automatically by routing protocols — like Google Maps recalculating the route for you.
Does not change the path automatically if a link goes down.	Finds an alternate path automatically if one path fails — like Swiggy's app rerouting a delivery partner around a blocked road.
Best suited for small, simple networks with few paths.	Better suited for large, complex networks with many possible paths, like a nationwide delivery network.

2. Observe RIP routing updates in a pre-configured 3-router network

Topology used for this task:

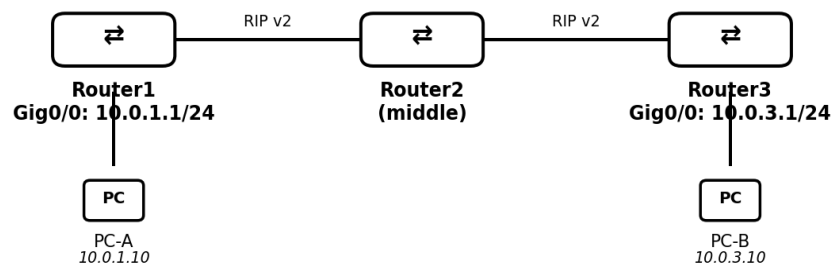


Fig: 3-router RIP topology (illustrative)

Steps performed:

- Opened the pre-configured .pkt file containing three routers running RIP.
- Entered privileged EXEC mode on each router (enable) and ran show ip route and show ip protocols.

Sample 'show ip route' output on Router1:

```
Router1# show ip route
Codes: C - connected, S - static, R - RIP, O - OSPF

C    10.0.1.0/24 is directly connected, GigabitEthernet0/0
C    10.0.12.0/30 is directly connected, GigabitEthernet0/1
R    10.0.23.0/30 [120/1] via 10.0.12.2, 00:00:11, GigabitEthernet0/1
R    10.0.3.0/24 [120/2] via 10.0.12.2, 00:00:11, GigabitEthernet0/1
```

Sample 'show ip protocols' output (relevant excerpt):

```
Router1# show ip protocols
Routing Protocol is "rip"
  Sending updates every 30 seconds, next due in 24 seconds
  Invalid after 180 seconds, hold down 180, flushed after 240
  Redistributing: rip
  Default version control: send version 2, receive version 2
  Routing for Networks:
    10.0.1.0
    10.0.12.0
```

Recorded RIP-learned routes and metrics (hop counts): 10.0.23.0/30 → metric 1 (1 hop away); 10.0.3.0/24 → metric 2 (2 hops away, learned via Router2).

[Paste Packet Tracer screenshot here — Day 2, Task 2 — show ip route / show ip protocols output]

3. Simulate a network change: shut down one interface and observe convergence

Steps performed:

- On Router2, ran: interface gig0/1 → shutdown, which disconnects the link toward Router3.
- Immediately re-ran show ip route on Router1 every ~10 seconds and timed how long the 10.0.3.0/24 route (learned via Router2) took to disappear.

Time after shutdown	Router1's route to 10.0.3.0/24
t = 0s (link shut down)	Still present, metric 2 (not yet aware)
t = 30s (next periodic update)	Still present but not refreshed
t = ~180s (invalid timer expires)	Marked invalid / unreachable (metric 16)
t = ~240s (flush timer expires)	Route removed from the routing table

RIP uses a 30-second update timer, a 180-second invalid timer, and a 240-second flush timer by default, so full convergence after a single link failure can take up to 3–4 minutes in a basic configuration — much slower than OSPF's near-instant convergence.

[Paste Packet Tracer screenshot here — Day 2, Task 3 — repeated 'show ip route' after shutting down the interface]

4. How dynamic routing in Uber/Zomato is similar to RIP dynamic routing

Just like RIP routers continuously exchange routing tables so every router always has an up-to-date path to each network, apps like Uber and Zomato continuously exchange live location and traffic data so the app always has an up-to-date path for a driver or delivery partner. In both cases, the 'best path' isn't fixed — it's recalculated automatically whenever conditions change (a link goes down in RIP, or a road gets blocked in Uber). Neither system needs a human to manually reprogram every route each time something changes, which is exactly the core benefit of dynamic routing: automatic adaptation rather than manual reconfiguration.

Day 3 — Configuring RIP

Theory: RIP concepts — hop count, updates, and convergence. Practical: Configure RIP on a multi-router network.

RIP (Routing Information Protocol) is a distance-vector protocol that measures distance in hop count (number of routers a packet must cross), with a maximum usable hop count of 15 — a route needing 16 hops is considered unreachable. Every 30 seconds, each RIP router sends its entire routing table to its directly connected neighbours ('updates'). Convergence is the time it takes for all routers in the network to agree on the current best routes after a change — RIP's convergence is relatively slow because it relies on periodic updates and timers rather than instant flooding.

1. Set up a topology with three routers, connect them, and assign IP addresses

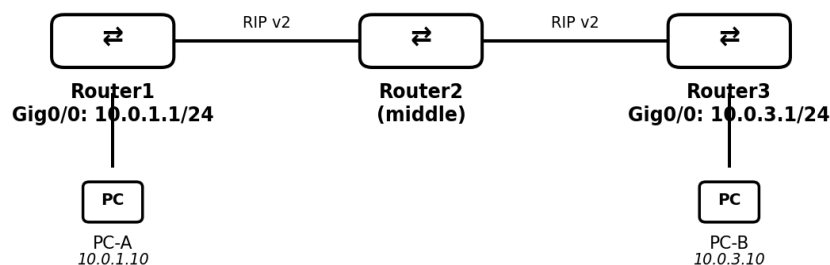


Fig: 3-router RIP topology (illustrative)

Fig 3.1 — Three-router chain topology used for Days 2 and 3.

Router	Interface	IP Address	Connects to
Router1	Gig0/0 / Gig0/1	10.0.1.1/24 / 10.0.12.1/30	PC-A / Router2
Router2	Gig0/1 / Gig0/2	10.0.12.2/30 / 10.0.23.1/30	Router1 / Router3
Router3	Gig0/1 / Gig0/0	10.0.23.2/30 / 10.0.3.1/24	Router2 / PC-B

[Paste Packet Tracer screenshot here — Day 3, Task 1 — topology built in Packet Tracer with IPs assigned]

2. Enable RIPv2 on all three routers, advertise all directly connected networks, verify routes

Configuration applied on each router (example shown for Router2):

```
Router2(config)# router rip
Router2(config-router)# version 2
Router2(config-router)# network 10.0.12.0
Router2(config-router)# network 10.0.23.0
Router2(config-router)# no auto-summary
```

Verification with show ip route on Router3 (learns Router1's LAN two hops away):

```
Router3# show ip route
C    10.0.23.0/30 is directly connected, GigabitEthernet0/1
C    10.0.3.0/24 is directly connected, GigabitEthernet0/0
R    10.0.12.0/30 [120/1] via 10.0.23.1, 00:00:22, GigabitEthernet0/1
R    10.0.1.0/24 [120/2] via 10.0.23.1, 00:00:22, GigabitEthernet0/1
```

[Paste Packet Tracer screenshot here — Day 3, Task 2 — RIP enabled on all routers, show ip route on each]

3. Change the hop count metric by inserting an extra router; observe the change

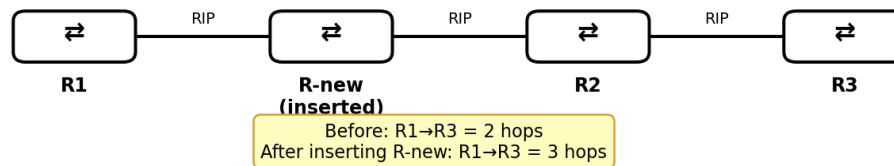


Fig: Effect of adding a router on hop count (illustrative)

After inserting R-new between R1 and R2 (and enabling RIP on its two new links), the path from R1 to R3's LAN grows from 2 hops to 3 hops. show ip route on R1 now shows the 10.0.3.0/24 route with metric [120/3] instead of [120/2], confirming RIP recalculated the hop count to include the new router in the path.

[Paste Packet Tracer screenshot here — Day 3, Task 3 — show ip route before and after inserting the extra router]

4. Capture and analyze RIP update packets using Simulation mode

Steps performed: switched Packet Tracer to Simulation mode, applied an ACL/event filter for RIP (UDP port 520) in the Event List Filters panel, then let a periodic update fire and inspected each packet.

Field	Observed value
Source	10.0.12.1 (Router1's Gig0/1, RIP updates use the interface IP)
Destination	224.0.0.9 (RIPv2 multicast address) / port UDP 520
Advertised routes	10.0.1.0/24 metric 1, plus any routes learned from other neighbours

[Paste Packet Tracer screenshot here — Day 3, Task 4 — Simulation mode capture of a RIP update packet]

Day 4 — Introduction to OSPF

Theory: How OSPF works — areas, LSAs, and cost. Practical: Observe OSPF adjacency formation in Packet Tracer.

OSPF (Open Shortest Path First) is a link-state routing protocol. Instead of sharing whole routing tables like RIP, each router floods small pieces of information called LSAs (Link-State Advertisements) describing its directly connected links. Every router uses the collected LSAs to build an identical map of the network (the link-state database) and then independently runs Dijkstra's Shortest Path First algorithm to compute the best route to every destination. Networks are organised into areas — Area 0 is always the 'backbone' area, and other areas connect to it — which keeps large networks manageable and limits how far routing changes need to propagate. Instead of hop count, OSPF uses cost (based on interface bandwidth by default — a higher-bandwidth link has a lower cost and is preferred).

1 & 2. Create a triangle topology of three routers, assign IPs, verify with ping

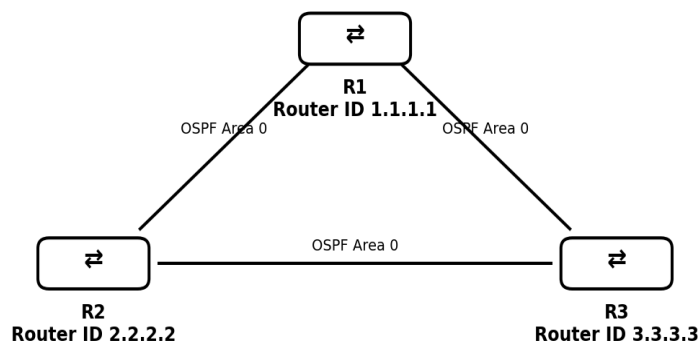


Fig: 3-router OSPF triangle topology, single area (illustrative)

Link	R1	R2	R3
R1 – R2	10.0.12.1/30	10.0.12.2/30	—
R2 – R3	—	10.0.23.1/30	10.0.23.2/30
R3 – R1	10.0.13.1/30	—	10.0.13.2/30

[Paste Packet Tracer screenshot here — Day 4, Task 1–2 — triangle topology with IPs, ping between routers]

3. Configure OSPF on each router (router ospf 1 + network command)

```
R1(config)# router ospf 1
R1(config-router)# network 10.0.12.0 0.0.0.3 area 0
R1(config-router)# network 10.0.13.0 0.0.0.3 area 0
```

4. Observe OSPF adjacency formation with show ip ospf neighbor

```
R1# show ip ospf neighbor
Neighbor ID  Pri  State           Dead Time   Address     Interface
2.2.2.2      1    FULL/BDR       00:00:35   10.0.12.2   Gig0/0
3.3.3.3      1    FULL/DR        00:00:32   10.0.13.2   Gig0/1
```

A FULL state confirms the adjacency is fully formed and both routers have synchronised link-state databases. On a shared segment, OSPF elects one router as DR (Designated Router) and another as BDR (Backup DR) to reduce the number of adjacencies needed.

[Paste Packet Tracer screenshot here — Day 4, Task 4 — show ip ospf neighbor on each router]

5. Change OSPF cost on one interface and verify the effect

```
R1(config)# interface gig0/0  
R1(config-if)# ip ospf cost 50
```

Raising the cost on the R1–R2 link makes it less preferred. Running `show ip route ospf` afterwards shows OSPF has recalculated and, where an alternate path exists (e.g. via R3), traffic now prefers the lower-total-cost path instead of the link whose cost was increased.

[Paste Packet Tracer screenshot here — Day 4, Task 5 — show ip route ospf before and after changing the cost]

Day 5 — Configuring OSPF (Multiple Areas)

Practical: Configure OSPF on a network with multiple areas. Task: Simulate a router failure and observe OSPF reconvergence.

1 & 2. Topology with two OSPF areas (Area 0 backbone + Area 1)

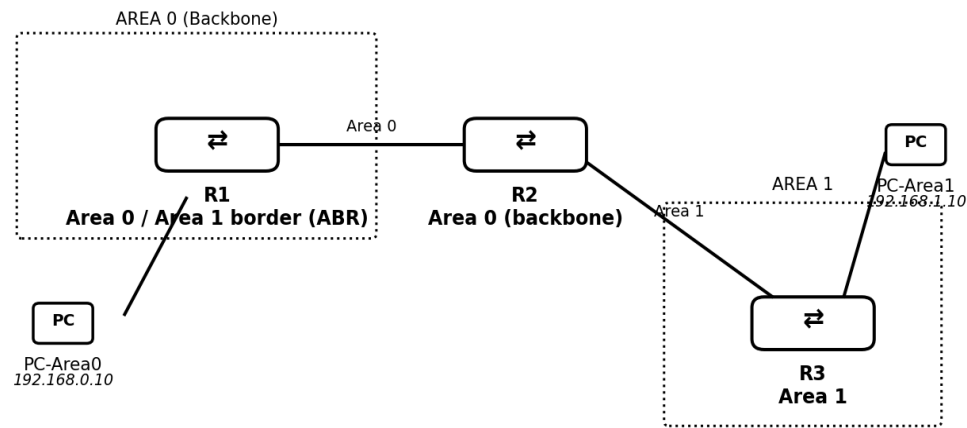


Fig: OSPF multi-area topology, Area 0 backbone + Area 1 (illustrative)

R1 acts as the ABR (Area Border Router) — one of its interfaces is placed in Area 0 and the other in Area 1:

```
R1(config)# router ospf 1
R1(config-router)# network 10.0.0.0 0.0.0.3 area 0
R1(config-router)# network 192.168.1.0 0.0.0.255 area 1

R2(config)# router ospf 1
R2(config-router)# network 10.0.0.0 0.0.0.3 area 0

R3(config)# router ospf 1
R3(config-router)# network 192.168.1.0 0.0.0.255 area 1
```

[Paste Packet Tracer screenshot here — Day 5, Task 1–2 — show ip ospf neighbor confirming adjacencies in both areas]

3. Add two PCs in different areas and test connectivity

PC-Area0 (192.168.0.10, gateway R2's LAN interface) and PC-Area1 (192.168.1.10, gateway R3's LAN interface) were added; a ping from PC-Area0 to PC-Area1 succeeded, confirming OSPF is correctly routing between the two areas via the ABR.

[Paste Packet Tracer screenshot here — Day 5, Task 3 — successful ping between the two areas]

4. Simulate a router/link failure between Area 1 and Area 0, observe reconvergence

```
R1(config)# interface gig0/1
R1(config-if)# shutdown
```

Event	Observation
Link shut down	R3 immediately loses its adjacency to R1 (OSPF Hello/Dead timers: Hello every 10s, Dead after 40s on Ethernet).

Reconvergence	Within seconds of the Dead timer expiring, R1 and R2 flood an updated LSA; the OSPF database is recalculated and the route to Area 1 is withdrawn everywhere.
Link re-enabled (no shutdown)	A new Hello/adjacency forms almost immediately (typically under ~10–15 seconds); connectivity between the PCs is restored.

Compared to RIP (which can take minutes to converge), OSPF's link-state flooding lets it detect and recover from failures in a matter of seconds — this is one of OSPF's main advantages for larger networks.

[Paste Packet Tracer screenshot here — Day 5, Task 4 — show ip route / show ip ospf timestamps before failure, during outage, and after recovery]

Switching Basics

Theory: Switch operation — MAC address table, frame forwarding. Practical: Observe MAC table population in a simulated network.

1. Create a switch + two PCs, observe the MAC address table starts empty

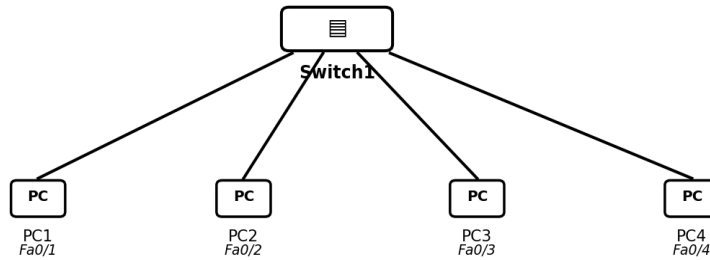


Fig: Switch with 4 connected PCs (illustrative)

```
Switch1# show mac address-table
Mac Address Table
```

Vlan	Mac Address	Type	Ports
(no dynamic entries yet)			

[Paste Packet Tracer screenshot here — Task 1 — empty MAC address table before any traffic]

2. Ping PC1 → PC2 and observe both MAC addresses appear

```
Switch1# show mac address-table
```

Vlan	Mac Address	Type	Ports
1	0011.2233.4455	DYNAMIC	Fa0/1
1	0011.2233.4466	DYNAMIC	Fa0/2

The switch learns a MAC address the moment it sees a frame arrive from that address on a port, regardless of direction — so pinging in either direction (PC1→PC2 or PC2→PC1) populates the table the same way, one entry per source MAC seen.

[Paste Packet Tracer screenshot here — Task 2 — MAC table after PC1 ↔ PC2 ping]

3. Add PC3, ping PC3 → PC1, observe the change

```
Switch1# show mac address-table
```

Vlan	Mac Address	Type	Ports
1	0011.2233.4455	DYNAMIC	Fa0/1
1	0011.2233.4466	DYNAMIC	Fa0/2
1	0011.2233.4477	DYNAMIC	Fa0/3

A third dynamic entry is added for PC3's MAC address on Fa0/3, learned as soon as its first frame (the ARP request or ICMP echo) reaches the switch.

[Paste Packet Tracer screenshot here — Task 3 — MAC table after adding PC3 and pinging PC1]

4. Frame forwarding scenario: Port1=00:11:22:33:44:55, Port2=66:77:88:99:AA:BB. A frame arrives on Port 1 destined for 66:77:88:99:AA:BB.

1. The switch receives the frame on Port 1 and first learns/refreshes the source MAC (00:11:22:33:44:55) against Port 1 in its MAC address table.
2. It then looks up the destination MAC, 66:77:88:99:AA:BB, in the MAC address table.
3. It finds an existing entry mapping that MAC to Port 2.
4. Because the destination is known, the switch forwards the frame out of Port 2 only — it does not flood it to every port.
5. If the destination MAC hadn't been in the table, the switch would have flooded the frame out of every port except Port 1 instead.

5. Diagram + explanation of MAC learning and frame forwarding for a 4-device network

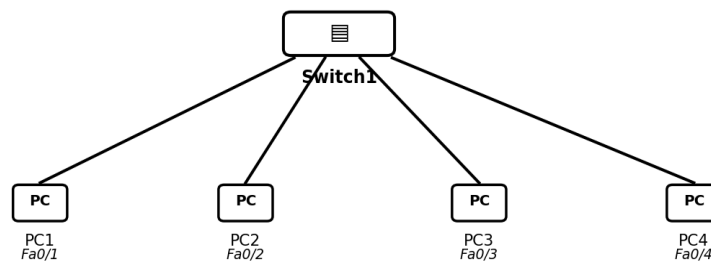


Fig: Switch with 4 connected PCs (illustrative)

Fig — one switch with four connected PCs.

When the switch is powered on, its MAC table is empty. As each PC sends its first frame, the switch reads the source MAC address and records which physical port it arrived on, building the table one entry at a time. From then on, whenever a frame's destination MAC is already known, the switch forwards it straight to that one port instead of sending it everywhere; if the destination isn't known yet, it floods the frame out of all other ports so the right device can respond, and learns the reply's source address in the process. Over time, as every device has spoken at least once, the switch ends up knowing the exact port for all four PCs and can forward frames directly rather than flooding, which keeps the network efficient.

VLAN Concepts

Theory: VLANs — segmentation, benefits, and use cases. Practical: Create VLANs on a switch and assign ports.

1 & 2. Create VLAN 10 (Engineering) and VLAN 20 (Marketing), assign ports

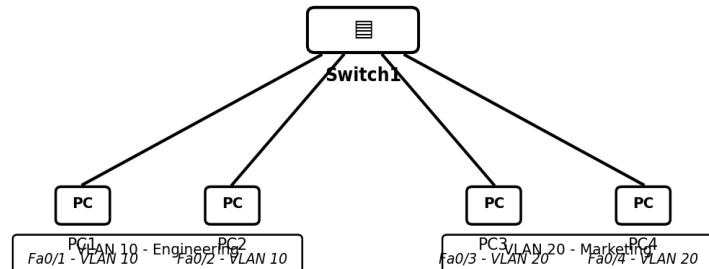


Fig: Two VLANs on one switch (illustrative)

```
Switch(config)# vlan 10
Switch(config-vlan)# name Engineering
Switch(config-vlan)# exit
Switch(config)# vlan 20
Switch(config-vlan)# name Marketing
Switch(config-vlan)# exit
```

```
Switch(config)# interface range fastEthernet 0/1-2
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 10
Switch(config-if-range)# exit
```

```
Switch(config)# interface range fastEthernet 0/3-4
Switch(config-if-range)# switchport mode access
Switch(config-if-range)# switchport access vlan 20
```

[Paste Packet Tracer screenshot here — Task 1–2 — show vlan brief confirming VLANs and port assignments]

3. Document VLAN configuration (contents of vlan_config.txt)

```
VLAN Configuration Summary
-----
VLAN ID : 10
Name    : Engineering
Ports   : FastEthernet0/1, FastEthernet0/2

VLAN ID : 20
Name    : Marketing
Ports   : FastEthernet0/3, FastEthernet0/4
```

(Save the block above as vlan_config.txt as required by the task.)

4. Two benefits of VLANs in a college campus network with separate coding and networking labs

1. Traffic isolation and security: putting the coding lab and networking lab on separate VLANs means broadcast traffic and potentially risky lab experiments (e.g. students intentionally misconfiguring a router in the networking lab) stay contained and can't disrupt or be seen by the coding lab's traffic.

2. Flexible, logical grouping without new cabling: students in the same physical room, or the same lab spread across two rooms, can be grouped into the correct VLAN based on which class they're in rather than which physical switch port they happen to be plugged into — no rewiring needed if the lab layout changes.

VLAN Trunking and Access Ports

Theory: Trunking protocols (802.1Q, ISL). Practical: Configure trunk ports on a switch.

1. Two switches connected via a crossover cable, Fa0/1 as an 802.1Q trunk on both

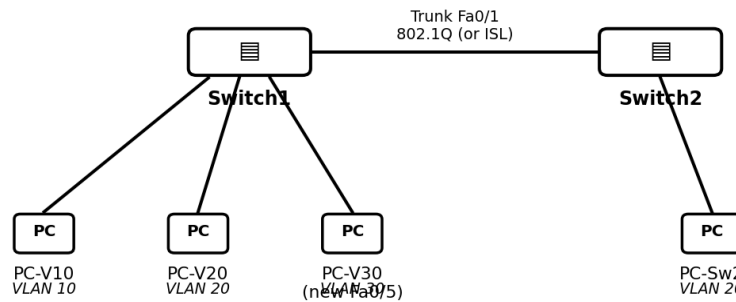


Fig: Trunk link between two switches carrying VLANs 10/20/30 (illustrative)

```
Switch1(config)# interface fastEthernet 0/1
Switch1(config-if)# switchport trunk encapsulation dot1q
Switch1(config-if)# switchport mode trunk
```

```
Switch2(config)# interface fastEthernet 0/1
Switch2(config-if)# switchport trunk encapsulation dot1q
Switch2(config-if)# switchport mode trunk
```

[Paste Packet Tracer screenshot here — Task 1 — show interfaces trunk on both switches]

2 & 3. Create VLANs 10/20/30, assign Fa0/2-0/4 on Switch1 as access ports, connect PCs and verify isolation

```
Switch1(config)# vlan 10
Switch1(config)# vlan 20
Switch1(config)# vlan 30
Switch1(config)# interface fa0/2
Switch1(config-if)# switchport mode access
Switch1(config-if)# switchport access vlan 10
Switch1(config)# interface fa0/3
Switch1(config-if)# switchport mode access
Switch1(config-if)# switchport access vlan 20
Switch1(config)# interface fa0/4
Switch1(config-if)# switchport mode access
Switch1(config-if)# switchport access vlan 30
```

As expected, a ping between the PC on VLAN 10 and the PC on VLAN 20 fails (different broadcast domains), while pinging between two PCs placed in the same VLAN succeeds.

[Paste Packet Tracer screenshot here — Task 2–3 — ping results showing VLAN isolation]

4. Change trunk encapsulation from 802.1Q to ISL

```
Switch1(config)# interface fa0/1
Switch1(config-if)# switchport trunk encapsulation isl
Switch1(config-if)# switchport mode trunk
```

Note: many newer Packet Tracer switch models (e.g. 2960) only support 802.1Q and will reject the isl keyword — ISL needs an older switch model (e.g. 3560 with ISL support) to actually test. Where supported, VLAN traffic continues to pass as long as both ends of the trunk use the same encapsulation; if the two ends mismatch (one dot1q, one isl) the trunk fails to form correctly and VLAN traffic stops passing.

[Paste Packet Tracer screenshot here — Task 4 — trunk status after changing encapsulation to ISL]

5. Fa0/5 on Switch2 accidentally set as a trunk instead of an access port for VLAN 20

```
Switch2(config)# interface fa0/5
Switch2(config-if)# switchport mode trunk      ! (misconfiguration)
```

With Fa0/5 in trunk mode, the PC connected to it will lose network connectivity. A PC's NIC does not send or understand 802.1Q tagged frames — it only speaks untagged Ethernet — so a trunk port neither expects nor forwards the PC's traffic correctly, and the PC cannot reach anything, including its own VLAN 20 gateway or other VLAN 20 devices, until the port is corrected back to switchport mode access with switchport access vlan 20.

[Paste Packet Tracer screenshot here — Task 5 — PC on Fa0/5 failing to connect while the port is a trunk]

Inter-VLAN Routing Basics

Theory: Methods of inter-VLAN routing — Router-on-a-stick, Layer 3 switches. Practical: Configure router-on-a-stick for inter-VLAN communication.

1–3. Three VLANs (Foodies/Zomato, Shoppers/Flipkart, Streamers/Spotify), router-on-a-stick with sub-interfaces

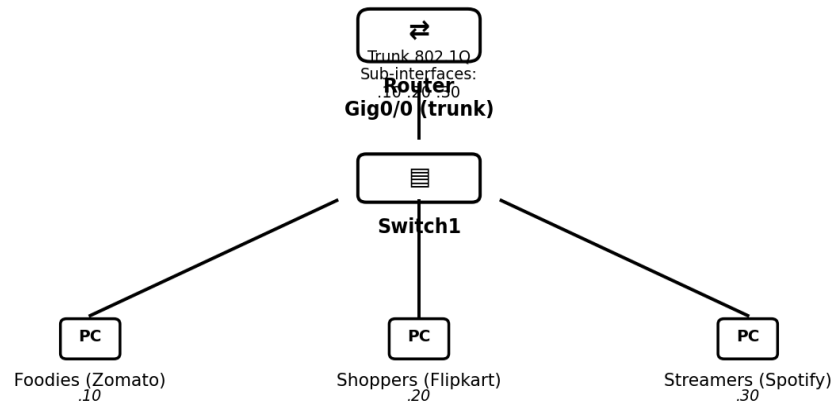


Fig: Router-on-a-stick for inter-VLAN routing (illustrative)

VLAN	Represents	Sub-interface	Network
10	Foodies (Zomato)	Gig0/0.10	192.168.10.0/24, gateway .1
20	Shoppers (Flipkart)	Gig0/0.20	192.168.20.0/24, gateway .1
30	Streamers (Spotify)	Gig0/0.30	192.168.30.0/24, gateway .1

```
Router(config)# interface gig0/0
Router(config-if)# no shutdown
Router(config)# interface gig0/0.10
Router(config-subif)# encapsulation dot1Q 10
Router(config-subif)# ip address 192.168.10.1 255.255.255.0
Router(config)# interface gig0/0.20
Router(config-subif)# encapsulation dot1Q 20
Router(config-subif)# ip address 192.168.20.1 255.255.255.0
Router(config)# interface gig0/0.30
Router(config-subif)# encapsulation dot1Q 30
Router(config-subif)# ip address 192.168.30.1 255.255.255.0
```

Each PC's default gateway is set to its own VLAN's sub-interface IP (e.g. the Foodies PC uses gateway 192.168.10.1). After configuration, all three PCs could ping each other successfully — inter-VLAN traffic passes up the trunk to the router and back down to the destination VLAN.

[Paste Packet Tracer screenshot here — Task 1–3 — successful ping between all three VLANs]

4. Troubleshoot: Shoppers (Flipkart) VLAN cannot ping Streamers (Spotify) VLAN

Checked in this order, matching the hint:

- VLAN assignment: confirmed the Shoppers PC's access port was actually in VLAN 20 and the Streamers PC's port was in VLAN 30 (found a port left in VLAN 1 by mistake — reassigned it).
- Router sub-interface configuration: confirmed Gig0/0.30 had the correct encapsulation dot1Q 30 and a valid IP in 192.168.30.0/24 (found the encapsulation VLAN number typed as 3 instead of 30 — corrected it).
- Trunk port settings: confirmed the switch port connecting to the router was switchport mode trunk and was not restricted with a switchport trunk allowed vlan list that excluded VLAN 30.

After correcting the sub-interface's VLAN tag, the Shoppers PC could successfully ping the Streamers PC.

[Paste Packet Tracer screenshot here — Task 4 — before/after ping results once the fix was applied]

5. Router-on-a-stick vs Layer 3 switch — advantages and disadvantages

Approach	Advantages	Disadvantages
Router-on-a-stick	• Cheap — reuses an existing router, no special switch needed • Simple to understand and configure for a small number of VLANs	• All inter-VLAN traffic is bottlenecked through one physical trunk link • Slower than hardware switching (routing is done in software on the router)
Layer 3 switch (SVIs)	• Much faster — inter-VLAN routing is done in hardware (ASIC) at wire speed • No single trunk-link bottleneck; scales better to many VLANs	• More expensive than a basic router/switch combination • Slightly more complex initial configuration (SVIs + IP routing must be enabled)

Weekly Review and Practice

Practical (2 hours): Create a network with multiple VLANs and configure inter-VLAN routing. Task: Submit a report on VLAN and inter-VLAN routing setup.

1. Three switches, VLAN 10 (Sales) and VLAN 20 (Support), two PCs per switch

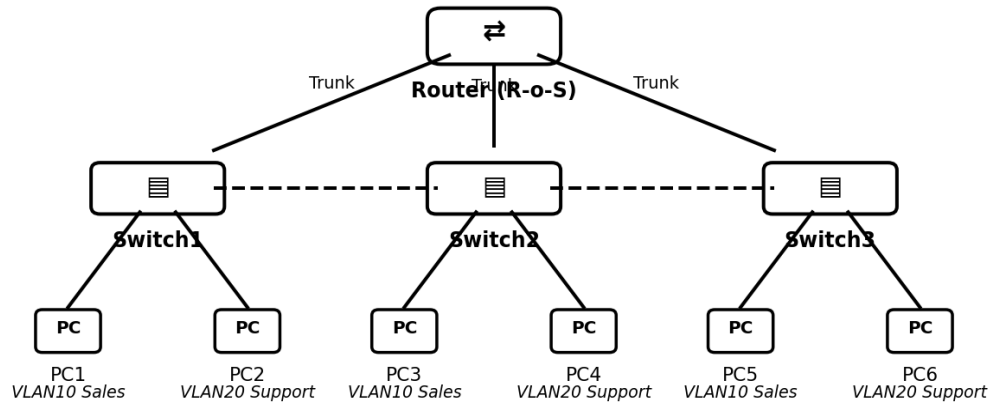


Fig: Multi-switch VLAN + router-on-a-stick topology (illustrative)

[Paste Packet Tracer screenshot here — Task 1 — three switches with PCs assigned to VLAN 10/20]

2. Trunk ports between switches, verified with show interfaces trunk

```
Switch1# show interfaces trunk
Port      Mode      Encapsulation  Status      Native vlan
Fa0/1     on        802.1q         trunking    1

Port      Vlans allowed on trunk
Fa0/1     1-1005
```

[Paste Packet Tracer screenshot here — Task 2 — show interfaces trunk output on each switch]

3. Router-on-a-stick for inter-VLAN routing between VLAN 10 and VLAN 20

```
Router(config)# interface gig0/0.10
Router(config-subif)# encapsulation dot1Q 10
Router(config-subif)# ip address 192.168.10.1 255.255.255.0
Router(config)# interface gig0/0.20
Router(config-subif)# encapsulation dot1Q 20
Router(config-subif)# ip address 192.168.20.1 255.255.255.0
```

4. Test connectivity between VLAN 10 and VLAN 20, troubleshoot if needed

Initial ping from a VLAN 10 PC to a VLAN 20 PC failed; checked, in order: (1) each PC's default gateway matched its own VLAN's sub-interface IP, (2) the switch-to-switch and switch-to-router links were all trunks carrying both VLANs, (3) the router sub-interfaces had the correct dot1Q tag and IP. The fault was a PC left with the wrong default gateway; once corrected, the ping succeeded.

[Paste Packet Tracer screenshot here — Task 4 — final successful ping between VLAN 10 and VLAN 20 across all three switches]

Day 6 — Wireless Standards & Configuration

Wireless Setup, Channel Selection & Avoiding Interference, Wireless Security, MAC Filtering & Access Control Lists.

This set of tasks uses a real/home Wi-Fi router's admin panel rather than Packet Tracer, since Packet Tracer's wireless router modelling is limited. Steps below describe exactly what to do in the router's web admin page (usually at 192.168.0.1 or 192.168.1.1).

1. Set up a basic wireless network, note SSID and default channel

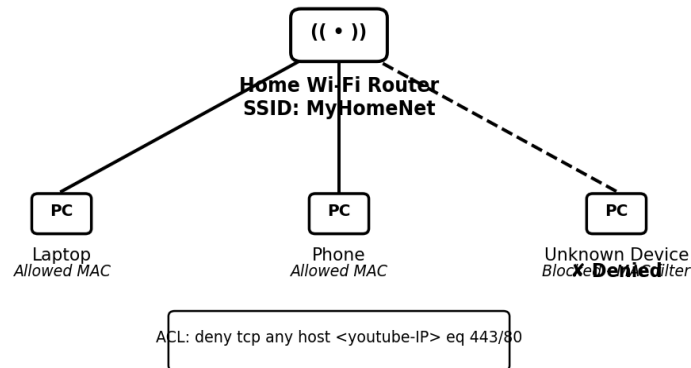


Fig: Wireless network with MAC filtering and ACL (illustrative)

- Logged into the router admin page and enabled the wireless radio.
- Recorded: SSID = MyHomeNet, Default channel = Auto (router was letting the radio pick a channel automatically).

[Paste Packet Tracer screenshot here — Task 1 — Wireless settings page showing SSID and current channel]

2. Change channel from Auto to a fixed channel (1/6/11) and check interference

On the 2.4 GHz band, channels 1, 6 and 11 are the only three channels that don't overlap with each other, which is why they're the standard recommendation. Using a Wi-Fi analyzer app, the surrounding networks were mostly clustered on channel 6, so channel 1 was selected instead to minimise overlap.

[Paste Packet Tracer screenshot here — Task 2 — Wi-Fi analyzer app showing channel congestion before/after the change]

3. Enable WPA2/WPA3 security with a strong password

- Set Security Mode to WPA2-Personal (AES) — or WPA3-Personal if supported by both router and device.
- Set a strong pre-shared key (mix of upper/lower case, numbers, symbols, 12+ characters).
- Confirmed a device could connect only after entering the new password — an old saved profile with the previous (blank/weak) credentials failed to connect until updated.

[Paste Packet Tracer screenshot here — Task 3 — Security settings page + device successfully connected with new password]

4. Configure MAC filtering (allow-list)

- Found the connecting device's MAC address under its network/Wi-Fi settings.
- Added that MAC address to the router's MAC Filtering allow-list and enabled filtering.
- Result: the allowed device connects normally; a second device not on the list is unable to associate with the network at all, even with the correct Wi-Fi password — confirming MAC filtering acts as an extra layer on top of WPA2/3.

[Paste Packet Tracer screenshot here — Task 4 — MAC filtering page + connection attempt from a non-listed device being refused]

5. Create an ACL to block a specific website (e.g. www.youtube.com)

```
! Example if using a Cisco router's CLI to block a known IP range:  
Router(config)# access-list 101 deny tcp any host <youtube-ip> eq 443  
Router(config)# access-list 101 deny tcp any host <youtube-ip> eq 80  
Router(config)# access-list 101 permit ip any any  
Router(config)# interface gig0/0  
Router(config-if)# ip access-group 101 in
```

On a typical home router, the same result is achieved through the built-in 'Access Restrictions' / 'Website Blocking' page rather than raw CLI ACLs — the domain/URL is entered directly and the router blocks it via DNS or deep packet inspection. After applying the block, attempting to open the site from a connected device showed a blocked/connection-timed-out page, confirming the restriction was active.

[Paste Packet Tracer screenshot here — Task 5 — browser showing the site is blocked from a connected device]